# Lankan Primire — Complete DevOps Architecture & Workflow Guide

> Interview preparation reference: architecture, CI/CD, Docker, Terraform, and monitoring explained A to Z.

---

## Table of Contents


1. Project Overview
2. System Architecture
3. Infrastructure as Code — Terraform
4. Containerization — Docker & Docker Compose
5. CI/CD Pipeline — GitHub Actions A to Z
6. Monitoring & Observability Stack
7. Security Implementation
8. Key Interview Questions & Answers


---

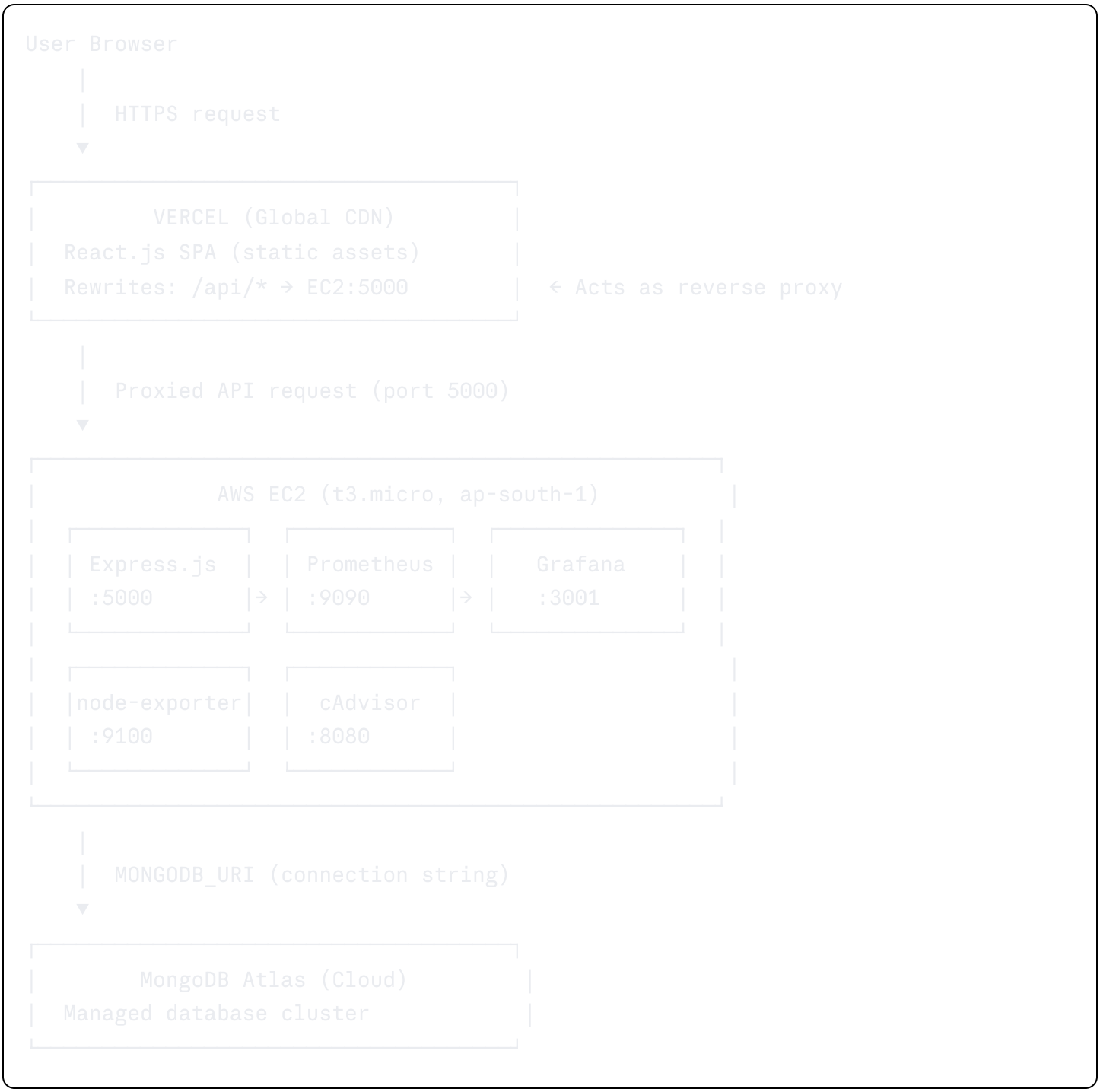
## 1. Project Overview

**Lankan Primire** is a full-stack movie ticketing application built with a **production-first DevOps mindset**. The entire infrastructure is automated — from provisioning cloud servers to deploying new code — with zero manual steps once set up.

| Layer | Technology | Where it runs |
|---|---|---|
| Frontend | React.js + Vite | Vercel (Global CDN) |
| Backend | Express.js (Node.js) | AWS EC2 (Docker) |
| Database | MongoDB | MongoDB Atlas (Cloud) |
| Proxy | nginx (inside Docker) | AWS EC2 |
| IaC | Terraform | Local → provisions AWS |
| CI/CD | GitHub Actions | GitHub cloud runners |
| Monitoring | Prometheus + Grafana | AWS EC2 (Docker) |

## 2. System Architecture

### 2.1 High-Level Flow




### 2.2 Why This Architecture?

**Vercel for frontend:** Serves the React app from edge locations worldwide. Users in Singapore, USA, and Europe all get fast load times from a nearby CDN node.

**Vercel Rewrites as reverse proxy:** The frontend makes API calls to `/api/movies`. Vercel rewrites this to `http://EC2_IP:5000/api/movies`. This solves the **Mixed Content** problem

(HTTPS frontend calling HTTP backend) without needing an SSL certificate on EC2.

**EC2 for backend:** Full control over the server environment. Docker runs all services in isolated containers on the same machine.

**MongoDB Atlas:** Managed cloud database — handles backups, replication, and scaling without manual DBA work.

---

## 3. Infrastructure as Code — Terraform

Terraform allows you to **define your entire AWS infrastructure in code** and create or destroy it with two commands:

```bash
terraform apply    # Creates everything
terraform destroy  # Tears everything down
```

### 3.1 What Terraform Creates in This Project

**File:** `infrastructure/terraform/main.tf`

#### Security Group (`aws_security_group`)

Acts as a cloud firewall. Opens exactly these ports:

| Port | Purpose | Access |
|---|---|---|
| 22 | SSH (your IP only) | Restricted (`var.allowed_ssh_ip`) |
| 80 | HTTP | Public (0.0.0.0/0) |
| 5000 | Express.js API | Public |
| 3000 | React frontend (nginx) | Public |
| 9090 | Prometheus | Public |
| 3001 | Grafana dashboard | Public |

> **Interview tip:** Opening port 9090/3001 publicly is fine for a showcase, but in production you'd restrict these to your IP only or put them behind a VPN.

#### EC2 Instance (`aws_instance`)

- **AMI:** Ubuntu 22.04 LTS (`ami-0f58b397bc5c1f2e8` — specific to `ap-south-1`)
- **Instance type:** `t3.micro` (1 vCPU, 1 GB RAM — free tier eligible)
- `user_data` **script:** Runs on first boot automatically. It:
  1. Installs Docker using the official convenience script
  2. Creates config files for Prometheus, Grafana datasources, and dashboards
  3. Writes `docker-compose.yml` to disk
  4. Runs `docker compose up -d` to start everything

**Elastic IP (`aws_eip`)**

Without this, EC2 gets a new public IP every time it reboots. The Elastic IP stays the same forever, so your Vercel rewrite URL never breaks.

### 3.2 Variable Flow

```
terraform.tfvars (never committed to git)
    │
    ├── key_pair_name      → SSH key for remote access
    ├── mongodb_uri        → Injected into docker-compose as env var
    ├── jwt_secret         → Injected into docker-compose as env var
    └── dockerhub_username → Which Docker Hub images to pull
```

**Output after `terraform apply`:**

```
ec2_public_ip  = "13.x.x.x"
frontend_url   = "http://13.x.x.x:3000"
backend_url    = "http://13.x.x.x:5000/api"
ssh_command    = "ssh -i my-key.pem ubuntu@13.x.x.x"
```

---

## 4. Containerization — Docker & Docker Compose

### 4.1 How Docker Works in This Project

Every service runs in its own Docker container. They are all connected via a private Docker network called `lankan-net`, so they can talk to each other by service name (e.g., `server:5000` instead of `localhost:5000`).

### 4.2 Server Dockerfile Explained

**File:** `server/Dockerfile`

dockerfile

```dockerfile
FROM node:20-alpine              # Minimal Node.js base image

WORKDIR /app

COPY package*.json ./
RUN npm ci                       # Install exact versions from lockfile

COPY . .
RUN rm -f .env                   # IMPORTANT: never bake secrets into image

EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=5s --start-period=30s --retries=3 \
  CMD wget -q --spider http://localhost:5000/api/health || exit 1

CMD ["node", "index.js"]
```

**Key decisions:**

- `npm ci` instead of `npm install` → reproducible builds, respects lockfile
- `rm -f .env` → secrets are NEVER baked into the image; injected at runtime
- `HEALTHCHECK` → Docker knows if the app crashed and can restart it

### 4.3 Client Dockerfile Explained (Multi-Stage Build)

**File:** `client/Dockerfile`

dockerfile

```
# ── Stage 1: Build ──────────────────────────────────
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL=/api
RUN npm run build              # Produces /app/dist (static HTML/JS/CSS)

# ── Stage 2: Serve ──────────────────────────────────
FROM nginx:alpine              # Tiny nginx image — no Node.js in final image!
COPY --from=builder /app/dist /usr/share/nginx/html
RUN echo 'server { ... }' > /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

**Why multi-stage?** The final image only contains nginx + static files. Node.js, npm, and all build tools are discarded. Result: the image is ~20MB instead of ~400MB.

**What the nginx config does:**

- `location /` → serves `index.html` (handles React Router client-side routing)
- `location /api` → proxies to `http://server:5000` (backend container)

### 4.4 Docker Compose — Production Stack

**File generated by CI/CD on EC2:**

yaml

```
services:
  server:          # Express.js API           → :5000
  client:          # nginx serving React       → :3000
  prometheus:      # Metrics collection        → :9090
  grafana:         # Dashboard visualization   → :3001
  node-exporter:   # Host machine metrics      → :9100
  cadvisor:        # Container metrics         → :8080

networks:
  lankan-net:
    driver: bridge  # All containers share this private network
```

**Container communication:**

```
client (nginx) → /api/* → server:5000
prometheus     → scrape → server:5000/metrics
prometheus     → scrape → node-exporter:9100
prometheus     → scrape → cadvisor:8080
grafana        → query  → prometheus:9090
```

---

## 5. CI/CD Pipeline — GitHub Actions A to Z

**File:** `.github/workflows/ci.yml`

### 5.1 Trigger Conditions

yaml

```yaml
on:
  push:
    branches: [ "main", "dev" ]      # Runs on push → triggers all jobs including de
  pull_request:
    branches: [ "main", "dev" ]      # Runs on PR → only test jobs (no deploy)
```

### 5.2 Complete Pipeline Flow

```
git push to main/dev
        |
        ▼
┌──────────────────────────────────┐
|  Job 1: backend-test  (parallel) |
|  Job 2: frontend-test (parallel) |
└──────────────────────────────────┘
        |  both must pass (needs: [])
        ▼
┌──────────────────────────────────┐
|  Job 3: docker-build             |
|  (only on push, not PR)          |
└──────────────────────────────────┘
        |  needs: docker-build
        ▼
┌──────────────────────────────────┐
|  Job 4: deploy                   |
```

```
|  (SSH into EC2 → docker compose)  |
```

### 5.3 Job 1 — Backend Tests

yaml

```yaml
backend-test:
  runs-on: ubuntu-latest
  defaults:
    run:
      working-directory: ./server    # All commands run inside /server

  steps:
    - uses: actions/checkout@v4       # Download repo code
    - uses: actions/setup-node@v4     # Install Node 20 with npm cache
      with:
        node-version: '20.x'
        cache: 'npm'
        cache-dependency-path: './server/package-lock.json'
    - run: npm ci                     # Install exact dependencies
    - run: npm test                   # Run test suite
      continue-on-error: true         # Don't fail pipeline if tests fail (showcase
```

### 5.4 Job 2 — Frontend Tests & Lint

yaml

```yaml
frontend-test:
  runs-on: ubuntu-latest
  defaults:
    run:
      working-directory: ./client

  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
    - run: npm ci
    - run: npm run lint               # ESLint check
      continue-on-error: true
    - run: npm run test -- --run      # Vitest (--run = no watch mode)
      continue-on-error: true
```

### 5.5 Job 3 — Build & Push Docker Images

This job only runs on `push` events (not PRs — you don't want to publish images for unmerged code).

yaml

```yaml
docker-build:
  needs: [backend-test, frontend-test]   # Waits for both test jobs
  if: github.event_name == 'push'

  steps:
    - uses: actions/checkout@v4

    - uses: docker/login-action@v3        # Authenticate with Docker Hub
      with:
        username: ${{ secrets.DOCKER_USERNAME }}
        password: ${{ secrets.DOCKER_PASSWORD }}

    - uses: docker/setup-buildx-action@v3 # Enable BuildKit (faster, better caching)

    - uses: docker/build-push-action@v5   # Build server image
      with:
        context: ./server
        push: true
        tags: |
          myuser/lankan-primire-server:latest
          myuser/lankan-primire-server:abc1234   # ← git SHA tag for rollback
```

**Two tags are pushed every time:**

- `:latest` → always points to newest version
- `:<git-sha>` → immutable snapshot, allows rollback to any previous commit

### 5.6 Job 4 — Deploy to EC2

This is the most important job. It SSHes into your EC2 instance and restarts the app.

yaml

```yaml
deploy:
  needs: docker-build
  if: github.event_name == 'push'

  steps:
    - uses: appleboy/ssh-action@v1.0.3
      env:
        MONGODB_URI: ${{ secrets.MONGODB_URI }}
        JWT_SECRET:  ${{ secrets.JWT_SECRET }}
      with:
        host:     ${{ secrets.EC2_IP }}
        username: ubuntu
        key:      ${{ secrets.SSH_PRIVATE_KEY }}
        port:     22
        script: |
          # 1. Write .env file (secrets never touch shell variables)
          printf "MONGODB_URI=%s\n" "$MONGODB_URI" > .env
          printf "JWT_SECRET=%s\n"  "$JWT_SECRET"  >> .env

          # 2. Write docker-compose.yml (full config recreated each deploy)
          cat <<EOF > docker-compose.yml
          services: ...
          EOF

          # 3. Pull new images and restart
          sudo docker compose pull
          sudo docker compose up -d --remove-orphans
          sudo docker image prune -f    # Clean up old images
```

**Why `printf` instead of `echo`?** `echo` can mangle special characters in passwords/URIs. `printf "%s"` is safe — it writes the value byte-for-byte.

**Why recreate `docker-compose.yml` on every deploy?** Ensures the running config always matches what's in the pipeline. No drift between what's on disk and what GitHub Actions expects.

### 5.7 GitHub Secrets Used

| Secret | Used in job | Purpose |
|---|---|---|
| `DOCKER_USERNAME` | docker-build | Log in to Docker Hub |
| `DOCKER_PASSWORD` | docker-build | Docker Hub password/token |

| Secret | Used in job | Purpose |
|---|---|---|
| `EC2_IP` | deploy | SSH target address |
| `SSH_PRIVATE_KEY` | deploy | Private key for EC2 SSH |
| `MONGODB_URI` | deploy | Database connection string |
| `JWT_SECRET` | deploy | JWT signing key |

## 6. Monitoring & Observability Stack

### 6.1 How the Metrics Flow

```
Express.js app
  |  exposes /metrics endpoint (prom-client library)
  ▼
Prometheus (scrapes every 15s)
  |  stores time-series data
  ▼
Grafana (queries Prometheus)
  |  renders dashboards
  ▼
You (view dashboards at :3001)
```

### 6.2 What Each Tool Monitors

**Prometheus scrape targets (`prometheus/prometheus.yml`):**

yaml

```yaml
scrape_configs:
  - job_name: 'api'         # Express.js app metrics
    targets: ['server:5000']
  - job_name: 'node'        # EC2 host machine (CPU, RAM, disk)
    targets: ['node-exporter:9100']
  - job_name: 'containers'  # Individual Docker container stats
    targets: ['cadvisor:8080']
```

**Grafana dashboards (`express-dashboard.json`) show:**

| Panel | PromQL Query | What it tells you |
|---|---|---|
| Request Rate | `rate(http_request_duration_seconds_count[5m])` | Requests/sec per route |
| Avg Latency | `rate(..._sum[5m]) / rate(..._count[5m])` | How slow your API is |
| Node.js Memory | `nodejs_memory_usage_bytes` | Heap/RSS memory of the app |

**node-exporter** → CPU %, RAM usage, disk I/O of the EC2 host **cAdvisor** → CPU/memory per Docker container (e.g. "grafana container uses 120MB RAM")

### 6.3 Grafana Setup (Auto-Provisioning)

Instead of clicking through the Grafana UI manually, everything is provisioned via files:

```
grafana/
  provisioning/
    datasources/
      prometheus.yml    → tells Grafana "connect to http://prometheus:9090"
    dashboards/
      provider.yml      → tells Grafana "load dashboards from this folder"
      express-dashboard.json  → the actual dashboard definition
```

On first startup, Grafana reads these files and is **fully configured automatically**.

---

## 7. Security Implementation

### 7.1 JWT Authentication

- Stateless session management — no server-side sessions
- User logs in → receives a signed JWT token
- Every subsequent request includes the token in the `Authorization: Bearer <token>` header
- Server verifies the signature using `JWT_SECRET`

### 7.2 Secret Management Strategy

| What | Where it's stored | How it reaches the app |
|---|---|---|
| MongoDB URI | GitHub Secrets | CI/CD writes it to `.env` on EC2 at deploy |

| What | Where it's stored | How it reaches the app |
|---|---|---|
| | | time |
| JWT Secret | GitHub Secrets | Same as above |
| Docker Hub password | GitHub Secrets | Used only during build job |
| SSH Private Key | GitHub Secrets | Used only during deploy job |
| Terraform secrets | `terraform.tfvars` (gitignored) | Local only, never committed |

**The `.env` file on EC2 is written fresh on every deployment.** It's never stored in the repo, never baked into a Docker image, and never printed in logs.

### 7.3 Role-Based Access Control (RBAC)

- Regular users access movie listings, seat selection, booking routes
- Admins access dashboard and management routes
- Middleware checks the JWT payload's `role` field before allowing access

### 7.4 SSH Security

- EC2 only allows SSH from `var.allowed_ssh_ip` (your IP in CIDR format)
- Deployment uses a dedicated SSH key pair — not your personal key
- The private key lives only in GitHub Secrets, never on disk

---

## 8. Key Interview Questions & Answers

**"What is CI/CD and how did you implement it?"**

CI/CD stands for Continuous Integration and Continuous Deployment. In this project, every `git push` to `main` automatically runs tests, builds Docker images, pushes them to Docker Hub, and deploys the new version to EC2 via SSH — all without any manual steps. I implemented this using GitHub Actions with four jobs: backend tests, frontend tests, Docker build+push, and SSH deployment.

**"What is Infrastructure as Code and why use it?"**

IaC means defining your cloud infrastructure in code files rather than clicking through a UI. I used Terraform to define the EC2 instance, security groups, and Elastic IP. The benefit is that the

entire production environment can be destroyed and recreated in minutes with `terraform apply`, and the configuration is version-controlled just like application code.

**"Why Docker? Why Docker Compose?"**

Docker ensures the app runs identically on any machine — my laptop, the CI runner, and the EC2 server. Docker Compose orchestrates multiple containers (app, Prometheus, Grafana, etc.) as a single stack, letting them communicate over a private network by service name rather than IP address.

**"What is a multi-stage Docker build?"**

A technique where the Dockerfile has two `FROM` statements. The first stage (builder) installs Node.js, installs dependencies, and compiles the React app. The second stage starts fresh with just nginx and copies only the compiled output. This keeps the final image small and free of build tools.

**"How do you handle secrets?"**

Secrets never touch the codebase. In CI/CD, they're stored as GitHub Secrets and injected into the EC2 `.env` file during deployment using `printf` (which is safe for special characters). In Terraform, they're stored in `terraform.tfvars` which is listed in `.gitignore`. Docker images are built without any secrets — they're injected at container runtime via `env_file`.

**"What monitoring do you have in place?"**

A three-tool observability stack: the Express.js app exposes a `/metrics` endpoint using `prom-client`. Prometheus scrapes this every 15 seconds along with host metrics (node-exporter) and container metrics (cAdvisor). Grafana visualizes all of this in real-time dashboards showing API request rates, average latency, CPU/RAM usage, and per-container memory footprints.

**"What happens if the app crashes on EC2?"**

All containers are started with `restart: unless-stopped` in Docker Compose, so Docker automatically restarts any crashed container. Additionally, each container has a `HEALTHCHECK` defined — Docker monitors this and considers the container unhealthy if the health endpoint stops responding, triggering a restart.

**"How does the frontend communicate with the backend?"**

The React app makes API calls to `/api/*` — a relative path. On Vercel, rewrite rules proxy these requests to the EC2 backend's public IP on port 5000. This approach avoids mixed content errors (HTTPS page calling an HTTP endpoint) since the request appears to the browser as same-origin.

**"What is an Elastic IP and why do you need one?"**

An AWS Elastic IP is a static public IP address that stays attached to your EC2 instance even if it's stopped and restarted. Without it, EC2 assigns a new random IP on every reboot, which would break the Vercel proxy rewrite URL and any DNS records pointing to the server.

**"Walk me through what happens on a git push."**

1. Developer pushes code to `main`
2. GitHub Actions triggers the workflow
3. Backend and frontend test jobs run in parallel on fresh Ubuntu VMs
4. If tests pass, the Docker build job starts — it builds both server and client images and pushes them to Docker Hub with `latest` and `<git-sha>` tags
5. The deploy job SSHes into EC2, writes a fresh `.env` file with secrets, regenerates `docker-compose.yml`, runs `docker compose pull` to get the new images, then `docker compose up -d` to restart containers with zero-downtime rolling replacement
6. Old images are pruned to save disk space

---

*Generated for Lankan Primire DevOps Interview Preparation*